

RoboWar Web

Robot Programmer's Guide

Stack-based assembly language reference for programming battle robots

Version 1.0 · Based on RoboWar 4.x

1. Introduction

RoboWar Web is a browser-based battle-robot simulator. You write programs in a simple stack-based assembly language — similar to Forth — that control an autonomous robot fighting inside a 2D arena. This guide covers everything you need to write, compile, and tune competitive robots.

How a battle works

A battle runs in discrete **ticks**. Each tick every living robot's program is executed for a fixed number of CPU **cycles** (set by its hardware). The program reads sensor registers, performs calculations on a stack, and writes to actuator registers to move, aim, fire, and shield. After all robots have run, the combat engine moves projectiles, resolves collisions, and applies damage. This repeats for up to 2 000 ticks; the last surviving robot (or the one with most armor remaining) wins.

Coordinate system: 0° points right (+X), angles increase clockwise (90° = down, 180° = left, 270° = up). The arena top-left is (0, 0).

2. The Stack Machine

Robot programs run on a **stack machine**. The stack holds 32-bit signed integers with a maximum depth of 256 entries (the oldest entry is silently dropped on overflow). Underflow returns 0 — programs never crash from an empty stack.

Key concepts

Concept	Description
Stack	Integer operand stack; max 256 values; underflow → 0
Registers	Named hardware I/O ports (read sensors / write commands)
Variables	100 numbered slots (1–100) for persistent per-tick storage
PC	Program counter persists between ticks — code runs continuously
Cycles	Max instructions executed per tick (set by CPU hardware level)

The PC (program counter) is **not reset** between ticks. Your program runs continuously across the entire battle — loops, subroutines, and conditional branches all work as expected.

3. Language Reference

3.1 Syntax basics

Programs are plain text. Tokens are separated by whitespace. Everything after a ; on a line is a comment.

```
; This is a comment
LOOP
  RADAR AIM      ; aim at nearest enemy
  1 FIRE         ; fire
```

3.2 Arithmetic & math operators

Token(s)	Stack effect	Description
42 / -7	$\rightarrow n$	Push integer literal
+ or ADD	$a\ b \rightarrow a+b$	Addition
- or SUB	$a\ b \rightarrow a-b$	Subtraction
* or MUL	$a\ b \rightarrow a*b$	Multiplication (32-bit)
/ or DIV	$a\ b \rightarrow a/b$	Integer division; $b=0 \rightarrow 0$
MOD	$a\ b \rightarrow a \bmod b$	Modulo (always non-negative)
ABS	$a \rightarrow a $	Absolute value
NEG	$a \rightarrow -a$	Negate
MAX	$a\ b \rightarrow \max(a,b)$	Larger of two values
MIN	$a\ b \rightarrow \min(a,b)$	Smaller of two values

3.3 Comparison operators

Comparisons push 1 (true) or 0 (false).

Token(s)	Stack effect	Description
= or EQ	$a\ b \rightarrow \text{flag}$	Equal
<> or NEQ	$a\ b \rightarrow \text{flag}$	Not equal
< or LT	$a\ b \rightarrow \text{flag}$	Less than
> or GT	$a\ b \rightarrow \text{flag}$	Greater than
<= or LTE	$a\ b \rightarrow \text{flag}$	Less than or equal
>= or GTE	$a\ b \rightarrow \text{flag}$	Greater than or equal

3.4 Boolean / bitwise operators

Token	Stack effect	Description
AND	$a\ b \rightarrow \text{flag}$	1 if both non-zero
OR	$a\ b \rightarrow \text{flag}$	1 if either non-zero
NOT	$a \rightarrow \text{flag}$	1 if zero, else 0
XOR	$a\ b \rightarrow a^b$	Bitwise XOR

3.5 Stack manipulation

Token	Stack effect	Description
DUP	$a \rightarrow a\ a$	Duplicate top
POP / DROP	$a \rightarrow$	Discard top
SWAP	$a\ b \rightarrow b\ a$	Swap top two
OVER	$a\ b \rightarrow a\ b\ a$	Copy second item to top
ROT	$a\ b\ c \rightarrow b\ c\ a$	Rotate top three: bottom to top

3.6 Variables

100 variable slots (numbered 1–100) persist across ticks for the robot's lifetime.

Token	Stack effect	Description
STORE n	value $\rightarrow$	Pop value, save to slot n (1–100)
RECALL n	$\rightarrow$ value	Push value from slot n

Use #DEFINE to give slots readable names:

```
#DEFINE target_angle 1    ; slot 1 holds last known bearing
#DEFINE low_energy 2     ; slot 2 is a flag

RADAR STORE target_angle ; save bearing
RECALL target_angle AIM  ; restore and aim
```

3.7 Control flow

Unconditional jump and subroutines

Token	Description
GOTO label	Unconditional jump to label
CALL label	Push return address, jump to label (max call depth 64)
RETURN	Pop return address, jump back (end of subroutine)
label:	Define a jump target (no instruction emitted)

Conditional — IF / ELSE / ENDIF

IF pops the top of the stack. If it is **0 (false)** execution jumps to the matching ELSE or ENDIF.

```
ENERGY 30 <
IF
    ; energy is low
    0 SHIELD
ELSE
    ; energy is fine
    1 SHIELD
ENDIF
```

Infinite loop — LOOP / POOL

LOOP marks the start of an infinite loop body; POOL jumps back unconditionally. Use GOTO to break out early.

```
LOOP
    ; code here runs every tick indefinitely
    1 FIRE
POOL
```

Blocks may be nested arbitrarily. The VM PC persists between ticks, so a LOOP body that takes many cycles to complete will resume mid-body on the next tick.

3.8 Macros with #DEFINE

#DEFINE *name value* replaces every occurrence of *name* with *value* at compile time. Works for number literals, variable slots, and hardware constants. Label operands after GOTO/CALL are **never** expanded.

```
#DEFINE FULL_SHIELD 1
#DEFINE SHIELD_SLOT 10
```

```
FULL_SHIELD STORE SHIELD_SLOT ; stores 1 in slot 10
```

4. Registers (Hardware Ports)

Reading a register pushes its current value onto the stack. Writing a register pops a value and sends a command to the hardware.

4.1 Sensor registers (read-only)

Register	Description
ENERGY	Current energy (0 – maxEnergy)
ARMOR	Current armor (0 – maxArmor); 0 means destroyed
HEAT	Current weapon heat (0 – 20); firing blocked at 20
RANGE	Distance to nearest enemy in radar cone (0 if none detected)
RADAR	Bearing in degrees to nearest enemy in radar cone
SPEEDX	Current horizontal velocity (negative = left)
SPEEDY	Current vertical velocity (negative = up)
POSX	Current X position in arena (pixels from left)
POSY	Current Y position in arena (pixels from top)
COLLISION	1 if robot collided with wall or another robot this tick, else 0
STUNNED	1 if robot is stunned this tick (hardware writes ignored)
TEAMMATES	Number of living teammates in team battles (0 in 1v1)
RANDOM	Random integer 0–255, seeded per-robot (deterministic)
TIME	Current tick number (0-based)

4.2 Actuator registers (write-only)

Register	Value	Description
SHIELD	0/1	Enable (1) or disable (0) energy shield
GUNX	integer	X component of aim direction (used with GUNY)
GUNY	integer	Y component of aim direction (used with GUNX)
AIM	degrees	Set aim angle directly in degrees (overrides GUNX/GUNY)
FIRE	0/1	Fire weapon this tick (1 = fire); ignored if heat ≥ 20 or no weapon
THRUSTX	-5..5	Horizontal thrust; clamped to ± 5 ; multiplied by engine accel
THRUSTY	-5..5	Vertical thrust; clamped to ± 5 ; multiplied by engine accel
BRAKE	0/1	Apply 20% velocity damping this tick (overrides thrust)
BEEP	any	No effect (reserved for sound in future versions)

Tip: RADAR gives the bearing to the nearest enemy. Writing that value to AIM points your gun at it instantly: RADAR AIM

GUNX / GUNY: If both are written this tick, the engine computes $\text{atan2}(\text{GUNY}, \text{GUNX})$ and uses that as the aim angle. Useful for offset aiming or predicting target movement.

5. Hardware System

Each robot has a **hardware budget of 30 HP**. Spend it across eight components. Unspent HP is wasted. Higher levels are strictly better but cost more.

5.1 Armor

Level	HP cost	Max armor
0	0	15
1	2	30
2	4	50
3	6	75
4	8	100

5.2 Shield

The shield reduces incoming damage but drains energy each tick it is active.

Level	HP cost	Energy drain/tick	Damage multiplier
0	0	0	1.00 (no effect)
1	2	2	0.75 (25% reduction)

Level	HP cost	Energy drain/tick	Damage multiplier
2	4	3	0.50 (50% reduction)
3	6	5	0.30 (70% reduction)

5.3 Weapon

Type	HP cost	Damage	Speed	Heat/shot	Notes
none	0	0	—	0	Unarmed
bullet	2	3	15	1	Unlimited ammo; fast; low heat
missile	4	8	8	3	Tracks nearest enemy for 5 ticks; bounces once off walls
drone	6	4	0	5	Stationary proximity mine; lasts 30 ticks
triple	6	3×3	15	4	3-bullet spread; hits multiple angles

Heat: Each shot adds heat. Firing is blocked when heat ≥ 20 . Heat dissipates each tick at the cooling dissipation rate.

5.4 Engine

Level	HP cost	Max speed	Accel
0	0	4	1
1	2	6	2
2	4	8	3
3	6	12	4

Actual velocity change per tick = $\text{THRUSTX/Y} \times \text{accel} \times 0.5$, clamped to $\pm \text{maxSpeed}$.

5.5 Energy

Level	HP cost	Max energy	Recharge/tick
0	0	50	1
1	2	100	2
2	4	150	3
3	6	200	4

5.6 CPU

Cycles executed per tick.

Level	HP cost	Cycles/tick
0	0	5
1	2	10

Level	HP cost	Cycles/tick
2	4	20
3	6	40

5.7 Cooling

Level	HP cost	Heat dissipation/tick
0	0	1
1	2	2
2	4	4

5.8 Radar

Radar detects the nearest enemy within its range and cone (centered on aim angle). A full-circle radar (level 3) detects enemies in any direction.

Level	HP cost	Range (px)	Cone
0	0	200	60°
1	2	350	120°
2	4	500	180°
3	6	999	360°

6. Programming Patterns

6.1 The minimal fighter

```
; Aims and fires continuously – no movement
LOOP
  RADAR AIM
  1 FIRE
POOL
```

Requires at least a bullet weapon and CPU level 1 for reliable execution. Works best with a 360° radar so enemies are always detected.

6.2 Energy-aware shield management

```
LOOP
  RADAR AIM
  1 FIRE
  ENERGY 40 > ; if energy > 40
  IF
    1 SHIELD ; keep shields up
  ELSE
```



```

        0 SHIELD      ; drop shields to recharge
    ENDIF
POOL

```

6.3 Simple chase-and-fire

```

; Chase the nearest detected enemy
LOOP
    RANGE 0 >      ; enemy detected?
    IF
        RADAR AIM          ; aim at enemy
        1 FIRE             ; fire
        RADAR 90 -         ; bearing minus 90 = perpendicular X
        THRUSTX            ; thrust toward enemy
        RADAR 90 <        ; if enemy is above
        IF
            -3 THRUSTY
        ELSE
            3 THRUSTY
        ENDIF
    ELSE
        0 THRUSTX          ; coast if no target
    ENDIF
POOL

```

6.4 Wall-bounce evasion

```

; Reverse direction when hitting a wall
LOOP
    RADAR AIM
    1 FIRE
    COLLISION
    IF
        SPEEDX NEG THRUSTX    ; reverse horizontal
        SPEEDY NEG THRUSTY    ; reverse vertical
    ELSE
        1 THRUSTX
        1 THRUSTY
    ENDIF
POOL

```

6.5 Subroutine pattern

```

; Main loop calls helper subroutines
LOOP
    CALL aim_and_fire
    CALL manage_shield
POOL

```

```

aim_and_fire:

```

```

RADAR AIM
HEAT 15 <
IF
    1 FIRE
ENDIF
RETURN

```

```

manage_shield:
    ENERGY 50 >
    IF
        1 SHIELD
    ELSE
        0 SHIELD
    ENDIF
    RETURN

```

Subroutines are called with CALL and return with RETURN. Max call depth is 64.

6.6 Using variables for state

```

#define last_bearing 1    ; remember last known enemy angle

LOOP
    RANGE 0 >
    IF
        RADAR DUP AIM      ; aim at enemy
        STORE last_bearing ; remember bearing
        1 FIRE
    ELSE
        RECALL last_bearing AIM ; keep aiming at last known position
    ENDIF
POOL

```

6.7 Missile robot

```

; Missile robots benefit from range – fire less often (high heat cost)
; but each hit deals 8 damage vs bullet's 3
LOOP
    RADAR AIM
    HEAT 5 <
    IF
        1 FIRE
    ENDIF
    ENERGY 60 > IF 1 SHIELD ELSE 0 SHIELD ENDIF
POOL

```

7. Common Mistakes & Tips

Mistake / Tip	Explanation
Forgetting FIRE pushes nothing	FIRE is a write-only register. Writing it does not consume the 1 from your stack — you must push
Stack accumulation in a loop	If you push values inside a loop without consuming them, the stack grows until it overflows (oldest
Overheating weapon	Each shot adds heat. With no cooling and a bullet (heat cost 1), you can fire ~20 times before HE
Shield drains energy fast	A level-3 shield drains 5 energy/tick. With level-0 energy (1 recharge/tick) your robot will quickly g
Radar cone misses enemies	Levels 0–2 have a limited cone centered on your aim angle. If your gun is not roughly pointing at t
LOOP / POOL is not a counted loop	LOOP/POOL creates an infinite loop with no count. To exit early, use GOTO.
IF consumes the top of stack	IF pops and tests the top value. If you need that value inside the IF body, DUP it before IF.
Hardware budget over 30 HP	The editor shows remaining HP. Going over 30 is not allowed — the robot will fail to save.

8. Quick Reference Card

All opcodes

Token	Stack effect	Token	Stack effect
n (literal)	$\rightarrow n$	DUP	$a \rightarrow a\ a$
ADD / +	$a\ b \rightarrow a+b$	POP/DROP	$a \rightarrow$
SUB / -	$a\ b \rightarrow a-b$	SWAP	$a\ b \rightarrow b\ a$
MUL / *	$a\ b \rightarrow a*b$	OVER	$a\ b \rightarrow a\ b\ a$
DIV / /	$a\ b \rightarrow a/b$	ROT	$a\ b\ c \rightarrow b\ c\ a$
MOD	$a\ b \rightarrow a\%b$	STORE n	$val \rightarrow$
ABS	$a \rightarrow a $	RECALL n	$\rightarrow val$
NEG	$a \rightarrow -a$	GOTO lbl	—
MAX	$a\ b \rightarrow \max$	CALL lbl	—
MIN	$a\ b \rightarrow \min$	RETURN	—
EQ / =	$a\ b \rightarrow 0/1$	IF	$flag \rightarrow$
NEQ / <>	$a\ b \rightarrow 0/1$	ELSE	—
LT / <	$a\ b \rightarrow 0/1$	ENDIF	—
GT / >	$a\ b \rightarrow 0/1$	LOOP	—
LTE / <=	$a\ b \rightarrow 0/1$	POOL	—
GTE / >=	$a\ b \rightarrow 0/1$	#DEFINE	(compile-time)
AND	$a\ b \rightarrow 0/1$	label:	(compile-time)
OR	$a\ b \rightarrow 0/1$		
NOT	$a \rightarrow 0/1$		
XOR	$a\ b \rightarrow a^b$		

All registers

Register (R)	R/W	Register (R/W)	R/W
ENERGY	R	SHIELD	W
ARMOR	R	GUNX	W
HEAT	R	GUNY	W
RANGE	R	AIM	W
RADAR	R	FIRE	W
SPEEDX	R	THRUSTX	W
SPEEDY	R	THRUSTY	W
POSX	R	BRAKE	W
POSY	R	BEEP	W (no-op)
COLLISION	R		
STUNNED	R		
TEAMMATES	R		
RANDOM	R		
TIME	R		

RoboWar Web is a browser-based recreation of Rod McFarland's RoboWar (1989–1994). Open the editor at **My Robots**, write your program, configure hardware, then head to **Battle** or **Tournament** to fight.